

# Frontend Technical Documentation — Public Service Reporter

The client application of the **public-service-reporter** project is designed as a Single Page Application (SPA) utilizing the **Vue 3** ecosystem and interactive Geospatial Information Systems (GIS). It is responsible for the intuitive reporting, geocoding, and visualization of municipal issues and infrastructure failures submitted by citizens directly to local government offices.

## 1. Technology Stack and Dependencies

- **Core Framework:** Vue 3.x (Composition API leveraging the modern `<script setup>` syntax)
- **Navigation & Routing:** Vue Router 4.x (complete with route metadata and role guards)
- **HTTP Client:** Axios (centralized network module, dynamic authentication headers)
- **Map Engine & GIS:**
  - `leaflet` (v1.9.4) — rendering of interactive map tiles and spatial layers.
  - `leaflet-control-geocoder` — geocoding utilities and address point lookups.
- **Build Environment:** Vue CLI (@vue/cli-service v5.x) / Node.js

## 2. Directory and Module Structure

```
frontend/
├── public/
│   └── index.html          # Base HTML template, Google Fonts integration
├── src/
│   ├── components/        # Reusable UI components and GIS map layers
│   │   ├── MapaPicker.vue # Interactive location selector (reporting form wizard)
│   │   ├── NavBar.vue     # Navigation and menu controller protected by Role-Based Access Control
│   │   └── ReportsMap.vue  # Aggregated reports map featuring the FlyTo animation engine
│   ├── router/            # Path configuration and private route protection
│   ├── services/
│   │   └── api.js         # Axios instance, reactive session states, and localStorage synchronization
│   ├── styles/
│   │   └── theme.css      # Design tokens, global color variables, and global styles
│   ├── App.vue            # Central layout container (App Shell & background decoration)
│   └── main.js             # Application bootstrapper and Vue 3 instance initialization
```

### 3. Visual Layer and Interface (UI/UX)

The application implements a dedicated graphic theme optimized for the high readability of technical-spatial data, incorporating modern web interface trends:

- **Typography (Google Fonts):**

- **Space Grotesk** : A geometric, sans-serif font used for headers, UI elements, status badges, and displaying GPS coordinates (\$Lat, Lng\$).
- **Spectral** : A classic, elegant serif typeface applied to report descriptions and official notices, reinforcing the public and authoritative nature of the platform.

- **Design System:**

- *Glassmorphism*: Semi-transparent interface components (e.g., the `.badge` class using `var(--surface-glass)` ) combined with thin, sophisticated borders.
- *Micro-animations*: Blurred decorative background orbs ( `.orb-1` and `.orb-2` ) utilizing radial gradients and a seamless `@keyframes float` loop to add depth.
- *Responsiveness*: Utilizing `clamp()` functions for flexible spacing control alongside Flexbox transitions that adapt gracefully on smaller viewports (<640px).

### 4. Authentication and State Synchronization (api.js)

The `src/services/api.js` module manages stateless token validation and front-end state persistence:

- **HTTP Client Isolation**: The Axios instance centralizes network layers. The API URL is resolved using `process.env.VUE_APP_API_BASE_URL` (defaulting to `http://localhost:5001` ).
- **Reactive State (Vue Refs)**: The service exports `isAuthenticated` , `currentUserId` , and `currentRole` flags, triggering immediate Virtual DOM updates across components upon session transitions.
- **Session Persistence**: Session tokens are robustly mirrored to `localStorage` through `storeToken` and `clearToken` lifecycle sequences.
- **Token Injection**: Upon initial boot, any stored token is instantly captured and attached as an `Authorization: Token <token>` header inside the shared Axios instance, aligning smoothly with the Django REST Framework.

## 5. Deep Dive into Map Components and Navigation

### A. MapPicker.vue — Interactive Spatial Selector

Embedded inside the report creation forms, this component mounts a Leaflet instance using clean *CartoDB Positron* map templates. It establishes real-time connections to the OpenStreetMap Nominatim API for bidirectional data synchronization:

1. **Geocoding:** Translates textual search parameters into geo-coordinates, panning the viewport camera to the exact location.
2. **Native W3C Geolocation:** Accesses device-level `navigator.geolocation` hardware keys to seamlessly query and pin the citizen's immediate hardware GPS coordinates.
3. **Reverse Geocoding:** Resolves map pin drops or hardware coordinates back into human-readable text addresses through background Nominatim callbacks, auto-populating form input elements.

### B. ReportsMap.vue — Aggregate Incident Visualizer

The core aggregate map engine used to present active incidents across the entire municipality. It maintains high rendering performance by tracking elements inside a `markerMap` cache object instead of repeatedly accessing the heavy DOM tree.

- **Watchers Engine:** The primary data watcher tracks changes within the reports array, pruning obsolete pins and mounting new entities. The secondary watcher handles the *Focus Engine* — when an item on the adjacent list is clicked, the specific marker changes its style to active ( `.map-pin--active` , applying a 1.5x CSS transform scale), and the map orchestrates a smooth flying motion ( `map.flyTo` ) before opening the interactive popup window.

### C. NavBar.vue — Role-Based Access Control (RBAC) Navigation

Maintains interface and view layer integrity. By subscribing to the reactive `currentRole` wrapper, it selectively mounts or drops administrative links ( `/admin` ) using the `v-if` directive, completely shielding privileged interfaces from normal citizens. It also safely manages token destruction sequences upon logout requests.



### Project Installation and Launch

To launch the application locally, run the following command sequences inside the frontend directory:

1. `npm install` (Installs project dependencies)
2. `npm run serve` (Launches local development server on port 8080 with hot-reload)
3. `npm run build` (Compiles and minifies assets to `/dist` for deployment pipelines)